

DOCUMENTO TÉCNICO

Proyecto Final – Arquitectura Serverless en AWS

PassVault

Materia: Arquitectura de sistemas en la Nube

Docente: Emilio Dorio

Carrera: Analista de Sistemas

Integrantes:

- Martin Cagliolo
- Estevez Pedro
- Fernando Feruglio

Fecha: 18 Junio de 2026

Índice

1. Introducción.....	3
1.1 Descripción del proyecto.....	3
1.2 Objetivo general.....	3
1.3 Objetivos específicos.....	3
2. Arquitectura del sistema.....	4
2.1 Arquitectura general.....	4
2.2 Funcionamiento de la arquitectura.....	5
2.3 Justificación de la arquitectura.....	5
3. Descripción de los componentes de la arquitectura.....	6
3.1 Amazon S3.....	6
3.2 Amazon CloudFront.....	6
3.3 Amazon API Gateway.....	7
3.4 AWS Lambda.....	7
3.5 Amazon DynamoDB.....	8
4. Flujo de funcionamiento del sistema.....	8
4.1 Guardar una contraseña.....	9
4.2 Consultar contraseñas.....	9
4.3 Eliminar una contraseña.....	9
5. Instrucciones de despliegue.....	10
5.1 Creación de la base de datos.....	10
5.2 Implementación de las funciones Lambda.....	10
5.3 Configuración de Amazon API Gateway.....	11
5.4 Publicación del frontend.....	11
5.5 Configuración de CloudFront.....	11
5.6 Verificación del sistema.....	12
6. Conclusiones.....	12

1. Introducción

1.1 Descripción del proyecto

PassVault es una aplicación web desarrollada bajo una arquitectura **Serverless** en Amazon Web Services (AWS), cuyo objetivo es permitir a los usuarios almacenar, consultar y eliminar contraseñas de manera sencilla mediante una interfaz web accesible desde cualquier navegador.

El sistema fue diseñado utilizando servicios administrados de AWS, eliminando la necesidad de configurar o mantener servidores propios. Esta arquitectura permite que cada componente cumpla una función específica y se comunique con el resto mediante servicios en la nube, logrando una solución escalable, de bajo costo y fácil de mantener.

El frontend está compuesto por una aplicación desarrollada con **HTML, CSS y JavaScript**, alojada en un bucket de Amazon S3 y distribuida a través de Amazon CloudFront para ofrecer acceso mediante HTTPS y mejorar los tiempos de carga. La comunicación con el backend se realiza utilizando solicitudes HTTP (`fetch()`), dirigidas a una API desarrollada con Amazon API Gateway.

La lógica del negocio se implementó mediante tres funciones AWS Lambda escritas en Python, responsables de guardar, consultar y eliminar registros. Finalmente, la información se almacena en una tabla de Amazon DynamoDB, una base de datos NoSQL completamente administrada por AWS.

De esta manera, el proyecto cumple con los principios de una arquitectura serverless, donde cada servicio realiza una tarea específica y la infraestructura es administrada automáticamente por el proveedor cloud.

1.2 Objetivo general

Desarrollar una aplicación web serverless para la gestión de contraseñas utilizando servicios de Amazon Web Services, implementando una arquitectura basada en **CloudFront, Amazon S3, API Gateway, AWS Lambda y Amazon DynamoDB**, permitiendo almacenar, visualizar y eliminar información mediante una interfaz web accesible desde Internet.

1.3 Objetivos específicos

- Diseñar un frontend estático utilizando HTML, CSS y JavaScript.
- Publicar la aplicación utilizando Amazon S3 y Amazon CloudFront.
- Implementar una API HTTP mediante Amazon API Gateway.
- Desarrollar funciones AWS Lambda en Python para procesar las operaciones del sistema.
- Almacenar la información utilizando Amazon DynamoDB.
- Integrar todos los componentes mediante una arquitectura completamente serverless.

- Demostrar el funcionamiento del sistema realizando operaciones de alta, consulta y eliminación de registros.

2. Arquitectura del sistema

2.1 Arquitectura general

La solución implementada se basa en una **arquitectura serverless**, donde la totalidad de la infraestructura utilizada es administrada por Amazon Web Services (AWS). En este modelo no fue necesario aprovisionar ni administrar servidores, ya que cada componente ejecuta una función específica y escala automáticamente según la demanda.

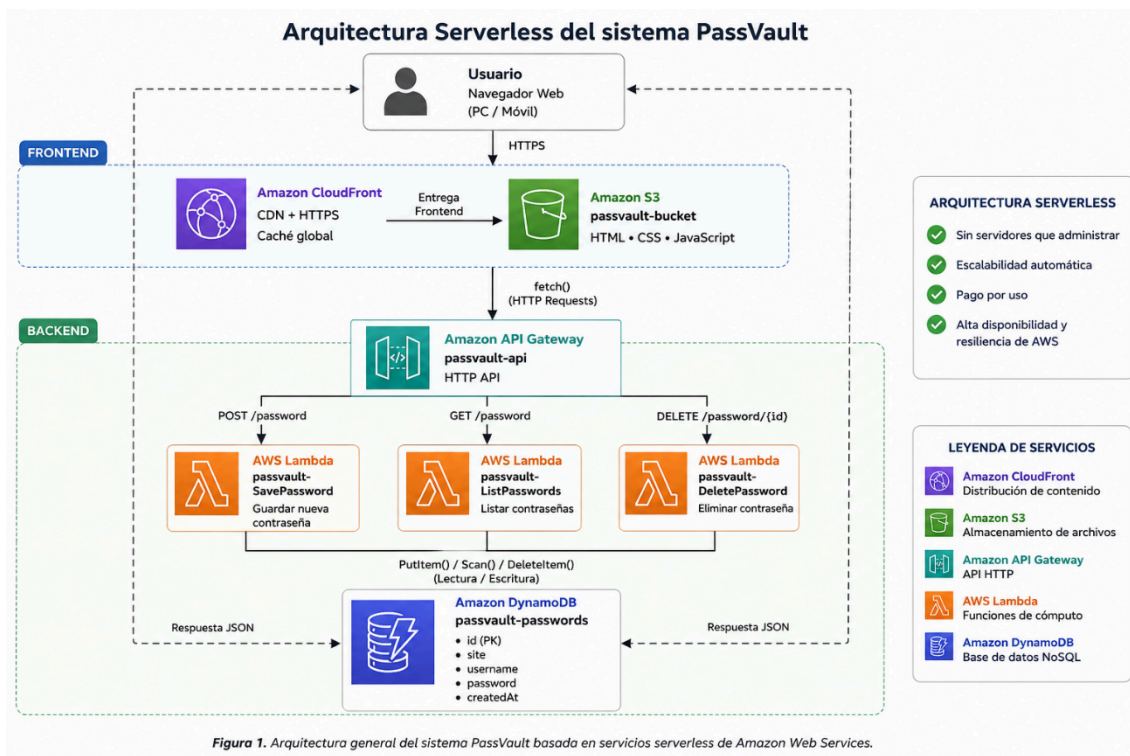
El sistema está compuesto por dos grandes capas:

- **Frontend**, encargado de la interacción con el usuario.
- **Backend**, responsable del procesamiento de las solicitudes y del acceso a la base de datos.

La comunicación entre ambos componentes se realiza mediante solicitudes HTTP enviadas desde el navegador hacia una API desarrollada con Amazon API Gateway. Esta API invoca las funciones AWS Lambda correspondientes, las cuales ejecutan la lógica del negocio y realizan las operaciones sobre Amazon DynamoDB.

La Figura 1 muestra la arquitectura implementada.

Figura 1. Arquitectura general de PassVault



2.2 Funcionamiento de la arquitectura

El funcionamiento del sistema comienza cuando el usuario accede a la aplicación desde un navegador web. La interfaz se encuentra almacenada en un bucket de Amazon S3 y es distribuida mediante Amazon CloudFront, permitiendo acceder al sitio utilizando una conexión segura HTTPS y reduciendo los tiempos de carga.

Una vez cargada la aplicación, todas las operaciones realizadas por el usuario son enviadas mediante solicitudes `fetch()` hacia Amazon API Gateway. Este servicio actúa como punto de entrada del backend y determina qué función Lambda debe ejecutarse según el método HTTP solicitado.

Cada función Lambda implementa una única responsabilidad:

- **passvault-SavePassword** recibe los datos ingresados por el usuario y almacena un nuevo registro en la base de datos.
- **passvault-ListPasswords** consulta la información almacenada y devuelve el listado completo en formato JSON.
- **passvault-DeletePassword** elimina un registro utilizando su identificador único.

Finalmente, las funciones Lambda interactúan con la tabla **passvault-passwords** de Amazon DynamoDB, donde se almacenan todos los datos de la aplicación.

La respuesta generada por Lambda es enviada nuevamente a través de API Gateway hasta el navegador, donde JavaScript actualiza la interfaz sin necesidad de recargar la página.

2.3 Justificación de la arquitectura

Se seleccionó una arquitectura **Serverless** debido a que responde adecuadamente a los requerimientos del proyecto y aprovecha las ventajas ofrecidas por AWS para aplicaciones de pequeña y mediana escala.

Las principales razones de esta elección fueron:

- **No requiere administrar servidores**, ya que AWS gestiona automáticamente la infraestructura.
- **Escalabilidad automática**, permitiendo que el sistema responda a un mayor número de usuarios sin realizar cambios en la infraestructura.
- **Pago por uso**, ya que los servicios consumen recursos únicamente cuando son utilizados.
- **Alta disponibilidad**, gracias a la infraestructura distribuida de AWS.
- **Separación de responsabilidades**, donde cada servicio cumple una función específica, facilitando el mantenimiento y la evolución del sistema.

Esta arquitectura permitió desarrollar una solución modular, sencilla de mantener y alineada con las buenas prácticas recomendadas para aplicaciones serverless.

3. Descripción de los componentes de la arquitectura

Cada componente de la solución cumple una función específica dentro de la arquitectura serverless implementada. La integración entre estos servicios permitió desarrollar una aplicación desacoplada, escalable y de fácil mantenimiento.

3.1 Amazon S3

Amazon S3 se utilizó para alojar el frontend de la aplicación PassVault, desarrollado con HTML, CSS y JavaScript. Al tratarse de un sitio web estático, este servicio ofrece una solución simple, confiable y altamente disponible para publicar los archivos que conforman la interfaz de usuario.

En este proyecto se creó el bucket **passvault-bucket**, donde se almacenan todos los archivos del frontend. El acceso directo al bucket permanece restringido, ya que el contenido es distribuido exclusivamente mediante Amazon CloudFront.

¿Por qué se eligió Amazon S3?

- Permite alojar sitios web estáticos de forma sencilla.
- Presenta alta disponibilidad y durabilidad.
- Se integra de manera nativa con CloudFront.
- Reduce costos al no requerir servidores web dedicados.

3.2 Amazon CloudFront

Amazon CloudFront actúa como red de distribución de contenido (CDN), permitiendo que los archivos almacenados en Amazon S3 sean entregados al usuario mediante una conexión HTTPS y con menores tiempos de respuesta.

En PassVault, CloudFront fue configurado para acceder al bucket privado utilizando **Origin Access Control (OAC)**, evitando que los archivos puedan descargarse directamente desde Amazon S3 y mejorando la seguridad del sistema.

Además de proporcionar acceso seguro, CloudFront almacena temporalmente los archivos más utilizados en sus ubicaciones distribuidas, reduciendo la latencia para los usuarios.

¿Por qué se eligió CloudFront?

- Permite utilizar HTTPS sin configuraciones adicionales.
- Mejora el rendimiento mediante caché.
- Incrementa la seguridad al proteger el acceso al bucket S3.
- Se integra completamente con Amazon S3.

3.3 Amazon API Gateway

Amazon API Gateway constituye el punto de entrada del backend. Su función consiste en recibir las solicitudes enviadas desde el frontend y dirigirlas hacia la función Lambda correspondiente según el método HTTP utilizado.

En este proyecto se implementó una **HTTP API** denominada **passvault-api**, la cual expone tres endpoints principales:

Método HTTP	Endpoint	Función
POST	/password	Guardar una contraseña
GET	/password	Obtener todas las contraseñas
DELETE	/password/{id}	Eliminar una contraseña

El frontend consume estos servicios utilizando la función `fetch()` de JavaScript y recibe las respuestas en formato JSON.

¿Por qué se eligió API Gateway?

- Facilita la creación de APIs REST sin administrar servidores.
- Se integra directamente con AWS Lambda.
- Permite administrar rutas, métodos HTTP y CORS desde un único servicio.
- Reduce la complejidad del backend.

3.4 AWS Lambda

AWS Lambda implementa la lógica de negocio del sistema. Cada operación fue desarrollada como una función independiente, siguiendo el principio de responsabilidad única, lo que facilita el mantenimiento y la escalabilidad del proyecto.

Las funciones implementadas fueron:

passvault-SavePassword

Recibe una solicitud HTTP POST con los datos de una contraseña, genera un identificador único y almacena el registro en Amazon DynamoDB.

passvault-ListPasswords

Consulta la tabla **passvault-passwords** y devuelve todas las contraseñas almacenadas en formato JSON para ser mostradas en la interfaz.

passvault-DeletePassword

Recibe el identificador de una contraseña mediante un parámetro de la URL y elimina el registro correspondiente de la base de datos.

Todas las funciones fueron desarrolladas en **Python** y cuentan con permisos para acceder a Amazon DynamoDB mediante un rol IAM.

¿Por qué se eligió AWS Lambda?

- No requiere administrar servidores.
- Ejecuta código únicamente cuando es invocada.
- Escala automáticamente según la demanda.
- Reduce costos al pagar únicamente por el tiempo de ejecución.
- Permite desarrollar funciones pequeñas y especializadas.

3.5 Amazon DynamoDB

Amazon DynamoDB es la base de datos utilizada para almacenar la información del sistema.

Se implementó una tabla denominada **passvault-passwords**, cuya clave primaria corresponde al atributo **id**. Cada registro contiene la información necesaria para identificar y administrar una contraseña almacenada por el usuario.

Los atributos definidos fueron:

Atributo	Descripción
----------	-------------

id	Identificador único del registro
----	----------------------------------

site	Sitio o servicio asociado
------	---------------------------

username	Nombre de usuario
----------	-------------------

password	Contraseña almacenada
----------	-----------------------

createdAt	Fecha y hora de creación
-----------	--------------------------

DynamoDB permite acceder a la información de forma rápida y sin necesidad de administrar servidores o motores de bases de datos tradicionales.

¿Por qué se eligió DynamoDB?

- Es un servicio completamente administrado.
- Presenta baja latencia en operaciones de lectura y escritura.
- Escala automáticamente según la demanda.
- Se integra de forma nativa con AWS Lambda.
- Resulta ideal para aplicaciones serverless.

4. Flujo de funcionamiento del sistema

El funcionamiento de PassVault se basa en la interacción entre el frontend, los servicios de AWS y la base de datos. Cada acción realizada por el usuario genera una solicitud HTTP que es procesada por la arquitectura serverless, devolviendo una respuesta inmediata a la aplicación.

A continuación se describen los principales flujos implementados.

4.1 Guardar una contraseña

Cuando el usuario completa el formulario y presiona el botón **Guardar**, la aplicación envía una solicitud HTTP **POST** hacia Amazon API Gateway.

API Gateway identifica la ruta solicitada e invoca la función **passvault-SavePassword**, la cual valida la información recibida, genera un identificador único para el registro y almacena los datos en la tabla **passvault-passwords** de Amazon DynamoDB.

Una vez finalizada la operación, Lambda devuelve una respuesta en formato JSON indicando que la contraseña fue almacenada correctamente. Finalmente, el frontend informa el resultado al usuario.

4.2 Consultar contraseñas

Al ingresar al sistema o actualizar la información, el frontend realiza una solicitud HTTP **GET** al endpoint correspondiente.

API Gateway redirige la solicitud hacia la función **passvault-ListPasswords**, la cual consulta todos los registros almacenados en DynamoDB mediante una operación **Scan()**.

Los datos obtenidos son enviados nuevamente al frontend en formato JSON y la aplicación actualiza automáticamente la lista de contraseñas sin necesidad de recargar la página.

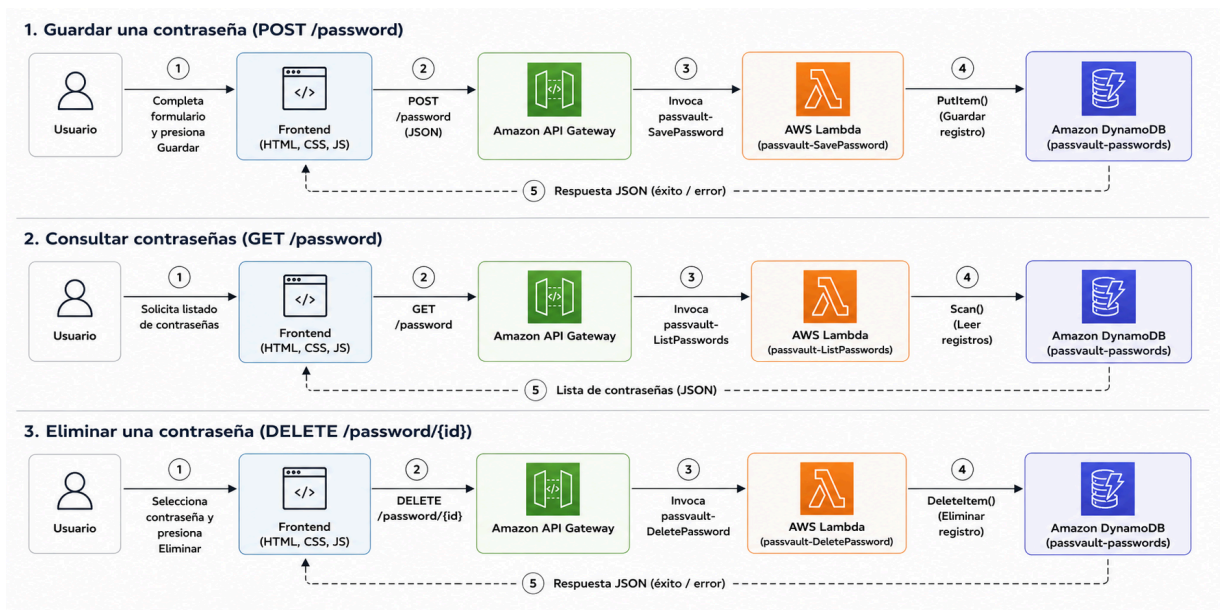
4.3 Eliminar una contraseña

Cuando el usuario selecciona la opción **Eliminar**, el frontend obtiene el identificador del registro y envía una solicitud HTTP **DELETE** hacia API Gateway.

La API invoca la función **passvault-DeletePassword**, que recibe el identificador como parámetro y ejecuta la operación **DeleteItem()** sobre DynamoDB.

Una vez eliminada la información, Lambda devuelve una confirmación al frontend, el cual actualiza automáticamente la lista de contraseñas mostrada al usuario.

Flujo de la operación



Este esquema evidencia la separación entre la interfaz de usuario, la lógica de negocio y el almacenamiento de datos, uno de los principios fundamentales de las arquitecturas serverless.

5. Instrucciones de despliegue

A continuación se describen los pasos necesarios para desplegar el sistema PassVault desde cero utilizando los servicios de Amazon Web Services (AWS).

5.1 Creación de la base de datos

1. Acceder al servicio **Amazon DynamoDB**.
2. Crear una nueva tabla denominada **passvault-passwords**.
3. Configurar como clave primaria el atributo **id** de tipo **String**.
4. Mantener la configuración predeterminada para el resto de las opciones.

Una vez creada la tabla, el sistema estará preparado para almacenar la información generada por las funciones Lambda.

5.2 Implementación de las funciones Lambda

Crear tres funciones AWS Lambda utilizando el runtime **Python**:

- **passvault-SavePassword**
- **passvault-ListPasswords**
- **passvault-DeletePassword**

Cada función debe:

- Incorporar el código correspondiente.
- Configurar el nombre de la tabla **passvault-passwords**.
- Asociar un rol IAM con permisos para acceder a Amazon DynamoDB.
- Realizar el despliegue (Deploy) y verificar su correcto funcionamiento mediante eventos de prueba.

5.3 Configuración de Amazon API Gateway

Crear una **HTTP API** denominada **passvault-api** e integrar las funciones Lambda mediante las siguientes rutas:

Método	Endpoint	Lambda
POST	/password	passvault-SavePassword
GET	/password	passvault-ListPasswords
DELETE	/password/{id}	passvault-DeletePassword

Posteriormente, habilitar la configuración **CORS** para permitir que el frontend pueda consumir la API desde el navegador.

5.4 Publicación del frontend

Crear un bucket de Amazon S3 denominado **passvault-bucket** y cargar los archivos correspondientes al frontend de la aplicación:

- HTML
- CSS
- JavaScript

El bucket debe permanecer privado y configurarse como origen de una distribución Amazon CloudFront mediante **Origin Access Control (OAC)**.

5.5 Configuración de CloudFront

Crear una distribución Amazon CloudFront utilizando como origen el bucket **passvault-bucket**.

Durante la configuración se debe:

- Utilizar Origin Access Control (OAC).
- Permitir únicamente el acceso a través de CloudFront.
- Esperar la finalización del despliegue antes de realizar las pruebas.

Una vez finalizada la distribución, CloudFront proporcionará una URL pública para acceder al sistema mediante HTTPS.

5.6 Verificación del sistema

Finalmente, acceder a la URL pública de CloudFront y comprobar el funcionamiento de las principales funcionalidades:

- Guardar una contraseña.
- Visualizar las contraseñas almacenadas.
- Eliminar una contraseña.
- Verificar que los cambios se reflejen correctamente en la tabla **passvault-passwords** de Amazon DynamoDB.

La correcta ejecución de estas pruebas confirma el funcionamiento integral de la arquitectura serverless implementada.

6. Conclusiones

El desarrollo de **PassVault** permitió aplicar de forma práctica los conceptos estudiados sobre arquitecturas serverless utilizando los principales servicios de Amazon Web Services.

La integración entre **Amazon S3, CloudFront, API Gateway, AWS Lambda y Amazon DynamoDB** permitió construir una aplicación web funcional sin necesidad de administrar servidores, aprovechando las ventajas de una infraestructura completamente administrada por AWS.

La separación de responsabilidades entre los distintos componentes facilitó el desarrollo y el mantenimiento del sistema. El frontend quedó desacoplado del backend, mientras que la lógica de negocio se implementó mediante funciones Lambda independientes y el almacenamiento de datos se centralizó en DynamoDB.

Además de cumplir con los requisitos establecidos para el proyecto, la solución desarrollada presenta características propias de las aplicaciones modernas basadas en la nube, como escalabilidad automática, alta disponibilidad, bajo costo operativo y facilidad de despliegue.

Como trabajo futuro, el sistema podría ampliarse incorporando funcionalidades como autenticación de usuarios mediante Amazon Cognito, cifrado de contraseñas, edición de registros, búsqueda avanzada e integración con otros servicios de AWS para mejorar la seguridad y la experiencia de uso.